Component-Level Design Document

Prometheus Financial Core — Loan Ledger Edition

Component-Level Design Document

Prepared by: Ledger & Solution Architect Agent
Date: August 16, 2026
Version: 1.0 (Draft)
Inputs: 8 approved epics / 61 REQ-IDs from the Loan Ledger BRD

Status: Design Scaffold, Not Implementation-Ready.
This document translates the approved BRD requirements into module ownership, interaction sequences, spec/event/flag inventories, and NFR notes. No external "Sections 4-8" architecture document was supplied — the constraints below (from the Architect Agent's system prompt) serve as the governing reference. Full OpenAPI 3.1 / AsyncAPI 2.6 contracts still need to be authored per the spec-file list in each section before implementation begins.

Architecture Context & Conventions

Modules

- **PTY** — Party/CIF
- **CRM** — CRM
- **GL** — General Ledger & Posting Engine
- **LAS** — Loan Account Subledger
- **PAY** — Payment Execution
- **BEOD** — Batch/EOD & Reconciliation

Non-Negotiable Constraints

- Each module owns its own data store; cross-module reads use published domain events or a synchronous OpenAPI call to the owning module — never direct database access.
- CRM has no path into the Posting Engine at all. If a design routes a CRM capability through GL, that is a design error, not a shortcut.
- The General Ledger & Posting Engine is the ONLY component permitted to write a JournalEntry. No other module may propose a balance mutation that bypasses it.
- Every JournalEntry is balanced (debits = credits), atomic, immutable, and idempotent (keyed on sourceEventId). There is no update or delete path.
- Every synchronous interface is an OpenAPI 3.1 contract; every asynchronous interface is an AsyncAPI 2.6 contract, before implementation.
- Every capability affecting money movement is gated by a feature flag via FeatureFlagClient, with both an engineering owner and a finance/risk owner named.

Shared Conventions Used Throughout

- GL exposes one generic endpoint, `POST /journal-entries` (idempotent on `sourceEventId`), extended per epic with new posting-rule codes (`PR-<EPIC>-<NN>`) — never a bespoke posting endpoint per epic.
- Spec files live under `/specs/openapi/*.yaml` (sync) and `/specs/asyncapi/*.yaml` (async).
- Cross-module reads are either a synchronous OpenAPI call to the owning module, or a subscription to that module's published event — never a direct database read.

Document Coverage

8 design notes · 46 interaction-sequence steps · 11 new posting rules · 24 spec-file changes · 17 domain events · 11 feature flags.

Table of Contents

Loan Account Booking [CB-BOOK]

1. Module Ownership

> LAS owns account creation and the term-version store. PTY is queried synchronously for party validation (not read directly). CRM and BEOD are event subscribers only. **GL is not invoked** — booking is intentionally a zero-posting epic.

2. Interaction Sequence

1. Origination system calls `LAS: POST /loan-accounts:book` (idempotent on `approvalReferenceId`) — REQ-BOOK-001, 004.
2. LAS synchronously calls `PTY: GET /parties/{partyId}` to confirm Active + KYC Verified — REQ-BOOK-002.
3. LAS creates the account (status Approved) with immutable term version v1 — REQ-BOOK-003, 006.
4. LAS publishes `LoanAccountBooked` — CRM logs an interaction (REQ-BOOK-005); BEOD tracks it for the approvals-vs-bookings check (REQ-BOOK-007).
5. **No Posting Engine call at any step** — a deliberate absence, not an oversight.

Posting-Rule Impact

No posting rules — this epic makes zero Posting Engine calls by design.

3. Spec Files (New / Modified)

File	Purpose
`/specs/openapi/loan-account-subledger.yaml`	add POST /loan-accounts:book
`/specs/asyncapi/loan-account-subledger-events.yaml`	new channel loan.account.booked.v1

4. Domain Events

Event	Producer	Consumer(s)
`LoanAccountBooked`	LAS	CRM, BEOD

5. Feature Flags

Key	Type	Rollout Plan	Owners

`loan.booking.enabled`	boolean kill-switch	Test-party dark launch → 10% of approval volume → 100%	Eng: Loan Subledger lead Risk: Not required — epic moves no money

6. NFR & Security Implications

- Synchronous PTY dependency needs a timeout + fail-closed circuit breaker (reject booking rather than silently skip KYC on timeout).
- Events carry partyId only, no PII payload.
- Term-version creation is logged with actor + approvalReferenceId for exam trail.
- Adds one new BEOD reconciliation job (approvals vs. bookings).

2 Loan Disbursement Processing [CB-DISB]

1. Module Ownership

> LAS owns the disbursement state machine and orchestrates PTY/GL/PAY calls. GL owns the funding posting. PAY owns payment execution. CRM/BEOD are subscribers.

2. Interaction Sequence

1. `LAS: POST /loan-accounts/{id}/disbursements` (idempotent on disbursementId), status → Pending Disbursement — REQ-DISB-001.
2. LAS → `PTY: GET /parties/{partyId}`; reject with reason code on failure — REQ-DISB-002.
3. LAS → `GL: POST /journal-entries`, rule **PR-DISB-01** (Dr LoanReceivable P / Cr Cash-Nostro P), sourceEventId=disbursementId — REQ-DISB-003.
4. On GL confirmation, LAS → `PAY: POST /payments:disburse` (idempotent on disbursementId) — REQ-DISB-004.
5. PaymentExecuted → LAS sets status Disbursed, publishes LoanAccountDisbursed → CRM (REQ-DISB-005), BEOD (REQ-DISB-006).
6. PaymentFailed/returned → LAS → GL posts **PR-DISB-02** (reversal of PR-DISB-01, new sourceEventId), status reverts to Approved only after GL confirms — REQ-DISB-007.

Posting-Rule Impact

Rule ID	Entry	Trigger
`PR-DISB-01`	Dr LoanReceivable P / Cr Cash-Nostro P	confirmed disbursement request, post party validation
`PR-DISB-02`	reversal of PR-DISB-01	confirmed payment failure/return

3. Spec Files (New / Modified)

File	Purpose
`/specs/openapi/loan-account-subledger.yaml`	add POST .../disbursements
`/specs/openapi/gl-posting-engine.yaml`	register PR-DISB-01, PR-DISB-02
`/specs/openapi/payment-execution.yaml`	add POST /payments:disburse
`/specs/asyncapi/payment-execution-events.yaml`	new channels payment.executed.v1, payment.failed.v1

4. Domain Events

Event	Producer	Consumer(s)
`JournalEntryPosted (Disbursement)`	GL	LAS, BEOD
`PaymentExecuted / PaymentFailed`	PAY	LAS
`LoanAccountDisbursed`	LAS	CRM, BEOD

5. Feature Flags

Key	Type	Rollout Plan	Owners
`loan.disbursement.enabled`	boolean kill-switch	Shadow-post (compute, no PAY call) 1 week → 5% live volume → 25% → 100%, auto-rollback on reversal-rate SLO breach	Eng: Loan Subledger lead Risk: Treasury Ops lead

6. NFR & Security Implications

- GL post target p95 < 200ms; PAY call can ack async (202) with status updates via event.
- Borrower account/routing details stay inside PAY, never in the disbursement event payload.
- Every state transition logged with correlationId=disbursementId.
- New BEOD daily total-variance check (REQ-DISB-006).

3 Daily Interest Accrual [CB-ACCR]

1. Module Ownership

> BEOD orchestrates the daily run; LAS computes eligibility and the accrual amount (using the term version in effect — see CB-MOD dependency); GL posts. No CRM touchpoint in this epic.

2. Interaction Sequence

1. BEOD triggers the accrual job for date D.
2. BEOD → `LAS: GET /loan-accounts:accrual-eligible?asOf=D` — REQ-ACCR-001, 005.
3. LAS computes X per account from rate/day-count/principal as of start-of-D, applying the payoff/closure cutoff — REQ-ACCR-004, 006.
4. LAS → `GL: POST /journal-entries`, rule **PR-ACCR-01** (Dr InterestReceivable X / Cr InterestIncome X), sourceEventId=accr:{loanAccountId}:{D} — REQ-ACCR-002, 003.
5. BEOD cross-checks eligible-account count vs. posted-entry count at batch close, raises AccrualExceptionRaised on mismatch — REQ-ACCR-007.

Posting-Rule Impact

Rule ID	Entry	Trigger
`PR-ACCR-01`	Dr InterestReceivable X / Cr InterestIncome X	daily eligible-account evaluation; idempotent on (loanAccountId, businessDate)

3. Spec Files (New / Modified)

File	Purpose
`/specs/openapi/loan-account-subledger.yaml`	add GET .../accrual-eligible
`/specs/openapi/gl-posting-engine.yaml`	register PR-ACCR-01
`/specs/asyncapi/batch-eod-events.yaml`	new channel accrual.exception.raised.v1

4. Domain Events

Event	Producer	Consumer(s)
`JournalEntryPosted (Accrual)`	GL	BEOD

`AccrualExceptionRaised`	BEOD	Ops/BEOD dashboard

5. Feature Flags

Key	Type	Rollout Plan	Owners
`loan.accrual.batch.enabled`	boolean kill-switch	Shadow-compute one full month-end cycle (no posting) → post for a pilot cohort → 100%	Eng: Batch/EOD lead Risk: Controller / Financial Reporting owner

6. NFR & Security Implications

- Must complete within the overnight window — partition eligible accounts, don't loop serially over the full portfolio.
- Every PR-ACCR-01 entry retains rate/day-count/principal as structured metadata for exam reproducibility (REQ-ACCR-004).
- This epic is itself a reconciliation control; no separate downstream recon needed.

4 Loan Repayment Processing [CB-REPAY]

1. Module Ownership

> PAY receives the inbound payment; LAS owns matching and waterfall allocation; GL posts; CRM logs.

2. Interaction Sequence

1. PAY publishes PaymentReceived (paymentReferenceId, amount, loanAccountRef).
2. LAS matches to exactly one account — REQ-REPAY-001; ambiguous/no match → PaymentUnmatched to BEOD's exception queue.
3. LAS computes waterfall (fees → interest → principal) — REQ-REPAY-002.
4. LAS → `GL: POST /journal-entries`, rule **PR-REPAY-01** (Dr Cash/Nostro P / Cr FeeReceivable + InterestReceivable + LoanReceivable per allocation — clearing receivables already recognized at accrual/assessment time, never re-crediting Income), idempotent on paymentReferenceId — REQ-REPAY-003, 004.
5. LAS publishes LoanRepaymentPosted → CRM — REQ-REPAY-005.
6. BEOD cross-checks all PaymentReceived against posted/unmatched outcomes — REQ-REPAY-006.
7. Reversal: `LAS: POST /repayments/{id}:reverse` → GL posts **PR-REPAY-02** (offsetting); misapplied case re-runs the waterfall against the correct account as a fresh PR-REPAY-01 with a new sourceEventId — REQ-REPAY-007.

Posting-Rule Impact

Rule ID	Entry	Trigger
`PR-REPAY-01`	Dr Cash/Nostro P / Cr FeeReceivable + InterestReceivable + LoanReceivable (per allocation)	matched, waterfall-allocated repayment — clears receivables built up by PR-ACCR-01/PR-DELINQ-01, never re-credits Income
`PR-REPAY-02`	reversal of PR-REPAY-01	confirmed return (NSF) or misapplication

3. Spec Files (New / Modified)

File	Purpose
`/specs/asyncapi/payment-execution-events.yaml`	new channel payment.received.v1
`/specs/openapi/loan-account-subledger.yaml`	add POST .../repayments/{id}:reverse

`/specs/openapi/gl-posting-engine.yaml`	register PR-REPAY-01, PR-REPAY-02

4. Domain Events

Event	Producer	Consumer(s)
`PaymentReceived`	PAY	LAS
`PaymentUnmatched`	LAS	BEOD
`LoanRepaymentPosted`	LAS	CRM

5. Feature Flags

Key	Type	Rollout Plan	Owners
`loan.repayment.autopost.enabled`	boolean kill-switch	Dual-run vs. legacy/manual posting 2 weeks (compare only) → cut over per payment rail, ACH first → all rails	Eng: Loan Subledger lead Risk: Cash Ops/Treasury lead

6. NFR & Security Implications

- Match→allocate→post should complete within seconds to hold same-day posting SLA.
- PaymentReceived carries reference IDs only, not payer bank details.
- Waterfall breakdown stored as line-item metadata for dispute resolution.
- Unmatched-payment queue needs its own alerting SLA, not just EOD visibility.
- Income-recognition principle: PR-REPAY-01 clears Cash against Fee/Interest/Loan Receivable — it never credits Fee/Interest Income directly, since that income was already recognized once at accrual (PR-ACCR-01) or assessment (PR-DELINQ-01) time. Crediting Income again here would double-count it.

5 Delinquency & Past-Due Fee Assessment [CB-DELINQ]

1. Module Ownership

> BEOD orchestrates; LAS owns DPD/fee computation and the Non-Accrual flag; GL posts; CRM logs.

2. Interaction Sequence

1. BEOD triggers the delinquency job for date D; LAS updates DPD buckets — REQ-DELINQ-001, 003.
2. Past-grace-period accounts: LAS → `GL: POST /journal-entries`, rule **PR-DELINQ-01** (Dr FeeReceivable / Cr FeeIncome) — REQ-DELINQ-002, 004.
3. DPD threshold crossing → LAS sets a Non-Accrual flag, read by **CB-ACCR step 2** as an eligibility filter — REQ-DELINQ-005 (cross-epic dependency, not a new posting).
4. LAS publishes DelinquencyStatusChanged → CRM — REQ-DELINQ-006.
5. BEOD cross-checks fee postings vs. DPD changes — REQ-DELINQ-008.
6. Waiver: `LAS: POST /fees/{id}:waive` with authorization metadata → GL posts **PR-DELINQ-02** (reversal) — REQ-DELINQ-007.

Posting-Rule Impact

Rule ID	Entry	Trigger
`PR-DELINQ-01`	Dr FeeReceivable / Cr FeeIncome	account past grace period on a missed installment
`PR-DELINQ-02`	reversal of PR-DELINQ-01	authorized fee waiver

3. Spec Files (New / Modified)

File	Purpose
`/specs/openapi/loan-account-subledger.yaml`	add GET .../past-due, POST /fees/{id}:waive
`/specs/openapi/gl-posting-engine.yaml`	register PR-DELINQ-01, PR-DELINQ-02
`/specs/asyncapi/loan-account-subledger-events.yaml`	new channels delinquency.status.changed.v1, loan.account.nonaccrual.v1

4. Domain Events

Event	Producer	Consumer(s)

`DelinquencyStatusChanged`	LAS	CRM
`LoanAccountNonAccrualFlagged`	LAS	LAS-internal (Accrual eligibility, CB-ACCR), BEOD

5. Feature Flags

Key	Type	Rollout Plan	Owners
`loan.delinquency.feeassessment.enabled`	boolean kill-switch	Shadow-compute 1 cycle → post for one cohort → all	Eng: Loan Subledger lead Risk: Collections/Risk lead
`loan.delinquency.feewaiver.enabled`	boolean, RBAC-gated	Enabled only for authorized Ops role from day one, no volume ramp	Eng: Loan Subledger lead Risk: Collections/Risk lead

6. NFR & Security Implications

- Same partitioning concern as Accrual.
- Waiver requires a mandatory (not optional) authorization record on PR-DELINQ-02.
- Sequencing dependency: CB-ACCR's daily step must run after this epic's Non-Accrual flag update in the BEOD DAG, or accrual uses stale eligibility.

6 Early Settlement / Loan Payoff [CB-PAYOFF]

1. Module Ownership

> LAS owns quote generation and closure; GL posts; PAY receives the payoff payment; CRM logs; BEOD reconciles.

2. Interaction Sequence

1. `LAS: GET /loan-accounts/{id}/payoff-quote?goodThrough=G` — read-only, no posting — REQ-PAYOFF-001, 002, 003.
2. Payoff payment arrives via PAY → PaymentReceived (reused from CB-REPAY), tagged with the quote reference.
3. LAS validates amount/window — exact match proceeds; underpayment routes to the standard waterfall (CB-REPAY reuse); overpayment excess routes to suspense — REQ-PAYOFF-003, 007.
4. LAS → `GL: POST /journal-entries`, rule **PR-PAYOFF-01** (Dr Cash/Nostro / Cr LoanReceivable + InterestReceivable, zeroing both) — REQ-PAYOFF-004, 005.
5. On confirmation, LAS sets status Closed — REQ-PAYOFF-006 — and publishes LoanAccountClosed → CRM (REQ-PAYOFF-008), BEOD.
6. BEOD confirms true zero balance on every account closed that day — REQ-PAYOFF-009.

Posting-Rule Impact

Rule ID	Entry	Trigger
`PR-PAYOFF-01`	Dr Cash/Nostro / Cr LoanReceivable + InterestReceivable (zeroing both)	payoff payment matching a valid, unexpired quote

3. Spec Files (New / Modified)

File	Purpose
`/specs/openapi/loan-account-subledger.yaml`	add GET .../payoff-quote
`/specs/openapi/gl-posting-engine.yaml`	register PR-PAYOFF-01
`/specs/asyncapi/loan-account-subledger-events.yaml`	new channel loan.account.closed.v1

4. Domain Events

Event	Producer	Consumer(s)

`LoanAccountClosed`	LAS	CRM, BEOD
`JournalEntryPosted (Payoff)`	GL	LAS, BEOD

5. Feature Flags

Key	Type	Rollout Plan	Owners
`loan.payoff.quote.enabled`	boolean, no money movement	Broad early enablement — read-only	Eng: Loan Subledger lead Risk: Not required — quote step posts nothing
`loan.payoff.settlement.enabled`	boolean, gates PR-PAYOFF-01 + closure	Ramp 10% → 50% → 100% of payoff volume	Eng: Loan Subledger lead Risk: Treasury/Controller

6. NFR & Security Implications

- Quote generation must be near-real-time (borrower/CSR-facing).
- Quote itemization is retained and linked to the settling journal entry for dispute traceability.
- Adds REQ-PAYOFF-009 to the BEOD daily job catalog.

7 Loan Charge-off & Write-off [CB-CHARGEOFF]

1. Module Ownership

> LAS owns the state machine and recovery orchestration; GL posts; CRM logs; BEOD reconciles the reserve account.

2. Interaction Sequence

1. Collections system calls `LAS: POST /loan-accounts/{id}:chargeoff` with an approved decision reference — REQ-CHGOFF-001, status Pending Charge-off.
2. LAS → `GL: POST /journal-entries`, rule **PR-CHGOFF-01** (Dr AllowanceForLoanLosses (B+I) / Cr LoanReceivable B, InterestReceivable I) — REQ-CHGOFF-002, 004.
3. On confirmation, status → Charged-Off — REQ-CHGOFF-003. LAS publishes LoanAccountChargedOff → CRM (REQ-CHGOFF-007), and this flag is also read by **CB-ACCR and CB-DELINQ's** eligibility queries to exclude charged-off accounts — REQ-CHGOFF-005 (same cross-epic pattern as the Non-Accrual flag).
4. Recovery: PAY → PaymentReceived (reused) → LAS matches to the charged-off account → GL posts **PR-CHGOFF-02** (Dr Cash/Nostro / Cr RecoveryIncome) — REQ-CHGOFF-006; status remains Charged-Off — this is **not** a reversal of PR-CHGOFF-01.
5. BEOD sums PR-CHGOFF-01 postings against the AllowanceForLoanLosses control-account balance — REQ-CHGOFF-008.

Posting-Rule Impact

Rule ID	Entry	Trigger
`PR-CHGOFF-01`	Dr AllowanceForLoanLosses (B+I) / Cr LoanReceivable B, InterestReceivable I	confirmed, approved charge-off decision
`PR-CHGOFF-02`	Dr Cash/Nostro / Cr RecoveryIncome	recovery payment on a charged-off account (distinct entry type, not a reversal)

3. Spec Files (New / Modified)

File	Purpose
`/specs/openapi/loan-account-subledger.yaml`	add POST .../chargeoff
`/specs/openapi/gl-posting-engine.yaml`	register PR-CHGOFF-01, PR-CHGOFF-02
`/specs/asyncapi/loan-account-subledger-events.yaml`	new channel loan.account.chargedoff.v1

4. Domain Events

Event	Producer	Consumer(s)
`LoanAccountChargedOff`	LAS	CRM, BEOD, LAS-internal (CB-ACCR & CB-DELINQ exclusion logic)
`JournalEntryPosted (ChargeOff/Recovery)`	GL	LAS, BEOD

5. Feature Flags

Key	Type	Rollout Plan	Owners
`loan.chargeoff.enabled`	boolean kill-switch	Pilot on one collections cohort with manual pre/post recon review each cycle → expand	Eng: Loan Subledger lead Risk: Controller / Allowance-for-Losses owner

6. NFR & Security Implications

- Not latency-sensitive (batch/manual-triggered process).
- chargeoffReferenceId must retain a durable link to the upstream credit-policy approval artifact.
- Reserve reconciliation aggregates across the entire portfolio against one GL control account — a different aggregation pattern than every other epic's per-account check; must be modeled distinctly in BEOD's job design.

8 Loan Modification (Rate/Term Change) [CB-MOD]

1. Module Ownership

> LAS owns term-versioning and the conditional posting branch; GL posts only for balance-impacting modifications; CRM logs.

2. Interaction Sequence

1. `LAS: POST /loan-accounts/{id}/modifications` (idempotent on modificationReferenceId) with new terms + effective date E — REQ-MOD-001, 005.
2. LAS appends immutable term version v_n+1 effective E; prior versions untouched — REQ-MOD-002.
3. **Branch A (capitalization):** LAS → `GL: POST /journal-entries`, rule **PR-MOD-01** (Dr LoanReceivable X / Cr InterestReceivable and/or FeeReceivable X) — REQ-MOD-003.
4. **Branch B (rate/term only):** no GL call is made — REQ-MOD-004; implemented as a true no-op, not a zero-amount journal entry (a zero-amount entry would still consume a sourceEventId and violate the “no false balance event” intent).
5. LAS publishes LoanTermsModified → CRM (REQ-MOD-007), and exposes `GET /loan-accounts/{id}/term-versions?asOf=D` — the read CB-ACCR must switch to for resolving “most recent version effective ≤ D” (REQ-MOD-006).
6. `GET /loan-accounts/{id}/term-versions` (full, unfiltered) serves the audit requirement — REQ-MOD-008.

Posting-Rule Impact

Rule ID	Entry	Trigger
`PR-MOD-01`	Dr LoanReceivable X / Cr InterestReceivable and/or FeeReceivable X	approved modification that capitalizes past-due interest/fees (conditional — Branch A only)

3. Spec Files (New / Modified)

File	Purpose
`/specs/openapi/loan-account-subledger.yaml`	add POST .../modifications, GET .../term-versions
`/specs/openapi/gl-posting-engine.yaml`	register PR-MOD-01
`/specs/asyncapi/loan-account-subledger-events.yaml`	new channel loan.terms.modified.v1

4. Domain Events

Event	Producer	Consumer(s)
`LoanTermsModified`	LAS	CRM, LAS-internal (CB-ACCR term-resolution)
`JournalEntryPosted (Modification)`	GL	LAS, BEOD — fires only for Branch A

5. Feature Flags

Key	Type	Rollout Plan	Owners
`loan.modification.enabled`	boolean, gates term-versioning broadly (Branch B has no money movement)	Enable rate/term-only path on a small cohort first; validate accrual correctly switches versions across the effective-date boundary before touching Branch A	Eng: Loan Subledger lead Risk: Not required for the Branch-B path
`loan.modification.capitalization.enabled`	boolean, gates PR-MOD-01 specifically	Ramp 10% → 50% → 100% of approved capitalization modifications, with reconciliation review each step	Eng: Loan Subledger lead Risk: Controller

6. NFR & Security Implications

- Not latency-sensitive.
- Term-version history must be retained indefinitely with no purge/TTL — confirm explicitly against the generic operational-log retention policy, which may default shorter.
- Breaking-change dependency: CB-MOD changes what CB-ACCR reads (a single static rate record → a versioned lookup); sequence delivery so CB-MOD's read API ships and is validated before CB-ACCR is cut over to consume it.

Appendix A — Cross-Epic Dependency Note

Three epics — CB-DELINQ (Non-Accrual flag), CB-CHARGEOFF (Charged-Off flag), and CB-MOD (term-version lookup) — all modify what CB-ACCR reads at daily-batch time. None of this is a scope problem, but the BEOD DAG and LAS's internal read contracts need explicit sequencing: CB-DELINQ and CB-CHARGEOFF's flag updates must run before CB-ACCR's eligibility query in the daily batch order, and CB-MOD's term-version read API must ship and be validated before CB-ACCR is cut over to consume it instead of a single static rate record. This is a build-order constraint for engineering, not a scope gap.

Appendix B — Consolidated Posting-Rule Catalog

Every journal entry this design introduces, across all 8 epics. GL is the only writer for all of these.

Rule ID	Epic	Entry	Trigger
`PR-DISB-01`	CB-DISB	Dr LoanReceivable P / Cr Cash-Nostro P	confirmed disbursement request, post party validation
`PR-DISB-02`	CB-DISB	reversal of PR-DISB-01	confirmed payment failure/return
`PR-ACCR-01`	CB-ACCR	Dr InterestReceivable X / Cr InterestIncome X	daily eligible-account evaluation; idempotent on (loanAccountId, businessDate)
`PR-REPAY-01`	CB-REPAY	Dr Cash/Nostro P / Cr FeeReceivable + InterestReceivable + LoanReceivable (per allocation)	matched, waterfall-allocated repayment — clears receivables built up by PR-ACCR-01/PR-DELINQ-01, never re-credits Income
`PR-REPAY-02`	CB-REPAY	reversal of PR-REPAY-01	confirmed return (NSF) or misapplication
`PR-DELINQ-01`	CB-DELINQ	Dr FeeReceivable / Cr FeeIncome	account past grace period on a missed installment
`PR-DELINQ-02`	CB-DELINQ	reversal of PR-DELINQ-01	authorized fee waiver
`PR-PAYOFF-01`	CB-PAYOFF	Dr Cash/Nostro / Cr LoanReceivable + InterestReceivable (zeroing both)	payoff payment matching a valid, unexpired quote
`PR-CHGOFF-01`	CB-CHARGEOFF	Dr AllowanceForLoanLosses (B+I) / Cr LoanReceivable B, InterestReceivable I	confirmed, approved charge-off decision

`PR-CHGOFF-02`	CB-CHARGEOFF	Dr Cash/Nostro / Cr RecoveryIncome	recovery payment on a charged-off account (distinct entry type, not a reversal)
`PR-MOD-01`	CB-MOD	Dr LoanReceivable X / Cr InterestReceivable and/or FeeReceivable X	approved modification that capitalizes past-due interest/fees (conditional — Branch A only)

Appendix C — Consolidated Domain Event Catalog

Event	Epic	Producer	Consumer(s)
`LoanAccountBooked`	CB-BOOK	LAS	CRM, BEOD
`JournalEntryPosted (Disbursement)`	CB-DISB	GL	LAS, BEOD
`PaymentExecuted / PaymentFailed`	CB-DISB	PAY	LAS
`LoanAccountDisbursed`	CB-DISB	LAS	CRM, BEOD
`JournalEntryPosted (Accrual)`	CB-ACCR	GL	BEOD
`AccrualExceptionRaised`	CB-ACCR	BEOD	Ops/BEOD dashboard
`PaymentReceived`	CB-REPAY	PAY	LAS
`PaymentUnmatched`	CB-REPAY	LAS	BEOD
`LoanRepaymentPosted`	CB-REPAY	LAS	CRM
`DelinquencyStatusChanged`	CB-DELINQ	LAS	CRM
`LoanAccountNonAccrualFlagged`	CB-DELINQ	LAS	LAS-internal (Accrual eligibility, CB-ACCR), BEOD
`LoanAccountClosed`	CB-PAYOFF	LAS	CRM, BEOD
`JournalEntryPosted (Payoff)`	CB-PAYOFF	GL	LAS, BEOD
`LoanAccountChargedOff`	CB-CHARGEOFF	LAS	CRM, BEOD, LAS-internal (CB-ACCR & CB-DELINQ exclusion logic)
`JournalEntryPosted (ChargeOff/Recovery)`	CB-CHARGEOFF	GL	LAS, BEOD

`LoanTermsModified`	CB-MOD	LAS	CRM, LAS-internal (CB-ACCR term-resolution)
`JournalEntryPosted (Modification)`	CB-MOD	GL	LAS, BEOD — fires only for Branch A

Appendix D — Consolidated Feature Flag Registry

Per the constraint, every flag gating money movement carries both an Engineering and a Finance/Risk owner. Flags marked "No" under Money Movement require Engineering ownership only.

Flag Key	Epic	Money Movement?	Engineering Owner	Finance/Risk Owner
`loan.booking.enabled`	CB-BOOK	No	Loan Subledger lead	Not required — epic moves no money
`loan.disbursement.enabled`	CB-DISB	Yes	Loan Subledger lead	Treasury Ops lead
`loan.accrual.batch.enabled`	CB-ACCR	Yes	Batch/EOD lead	Controller / Financial Reporting owner
`loan.repayment.autopost.enabled`	CB-REPAY	Yes	Loan Subledger lead	Cash Ops/Treasury lead
`loan.delinquency.feeassessment.enabled`	CB-DELINQ	Yes	Loan Subledger lead	Collections/Risk lead
`loan.delinquency.feewaiver.enabled`	CB-DELINQ	Yes	Loan Subledger lead	Collections/Risk lead
`loan.payoff.quote.enabled`	CB-PAYOFF	No	Loan Subledger lead	Not required — quote step posts nothing
`loan.payoff.settlement.enabled`	CB-PAYOFF	Yes	Loan Subledger lead	Treasury/Controller
`loan.chargeoff.enabled`	CB-CHARGEOFF	Yes	Loan Subledger lead	Controller / Allowance-for-Losses owner
`loan.modification.enabled`	CB-MOD	No	Loan Subledger lead	Not required for the Branch-B path
`loan.modification.capitalization.enabled`	CB-MOD	Yes	Loan Subledger lead	Controller